

03_catalogos_y_torneo_fuzzy

May 9, 2026

1 03. Catalogos y Torneo Fuzzy

1.1 Goal

Load SCVS and SRI catalogs once, then run the fuzzy tournament to select the best candidate name or RUC for unmatched companies.

1.2 Inputs

- Outputs from exact SCVS matching.
- 01_data_ingestion_enrichment/data_SRI/
- 01_data_ingestion_enrichment/data_super_compañias/

1.3 Outputs

- outputs/super_best.parquet
- outputs/sri_razon_best.parquet
- outputs/sri_fantasia_best.parquet
- outputs/leads_torneo_ganadores.csv
- outputs/horas_torneo_ganadores.csv

```
[1]: # Helpers y rutas
from pathlib import Path
import re, unicodedata, os
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings("ignore", category=UserWarning, module="openpyxl")

try:
    from IPython.display import display
except Exception:
    def display(x): print(x)

def find_project_root(start=None):
    start = (start or Path.cwd()).resolve()
    for p in [start, *start.parents]:
        if (p / "01_data_ingestion_enrichment").is_dir() and (p / "02_data_cleaning").is_dir():
```

```

        return p
    raise FileNotFoundError(f"No se pudo localizar la raíz. cwd: {start}")

def ensure_dir(d): Path(d).mkdir(parents=True, exist_ok=True); return Path(d)

def save_df_csv(df, path, *, index=False, encoding="utf-8-sig"):
    path = Path(path); ensure_dir(path.parent)
    df.to_csv(path, index=index, encoding=encoding)
    print(f"[OK] Guardado: {path.resolve()} shape: {df.shape}")
    return path

def read_csv_checked(path, **kwargs):
    path = Path(path)
    if not path.exists(): raise FileNotFoundError(f"No existe: {path.resolve()}")
    return pd.read_csv(path, **kwargs)

ROOT = find_project_root()
INGESTION_DIR = ROOT / "01_data_ingestion_enrichment"
CLEAN_DIR = ROOT / "02_data_cleaning"
CLEAN_OUT = CLEAN_DIR / "outputs"

# Buscar data_SRI en ambas ubicaciones posibles
_SRI_CANDIDATES = [CLEAN_DIR / "data_SRI", INGESTION_DIR / "data_SRI"]
SRI_DIR = next((p for p in _SRI_CANDIDATES if p.exists() and any(p.iterdir())) ,
    None)
if SRI_DIR is None:
    raise FileNotFoundError(f"No encontré data_SRI en ninguna de:
    {_SRI_CANDIDATES}")

# Buscar data_super_compañías en ambas ubicaciones posibles
_SCVS_CANDIDATES = [CLEAN_DIR / "data_super_compañías", INGESTION_DIR /
    "data_super_compañías"]
SCVS_DIR = next((p for p in _SCVS_CANDIDATES if p.exists()), None)
if SCVS_DIR is None:
    raise FileNotFoundError(f"No encontré data_super_compañías en ninguna de:
    {_SCVS_CANDIDATES}")

print(f"[CONFIG] ROOT : {ROOT}")
print(f"[CONFIG] SRI_DIR : {SRI_DIR}")
print(f"[CONFIG] SCVS_DIR : {SCVS_DIR}")
print(f"[CONFIG] CLEAN_OUT: {CLEAN_OUT}")

```

```

[CONFIG] ROOT : E:\TESIS MAESTRIA\Desarrollo_clustering_maestria
[CONFIG] SRI_DIR : E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\data_SRI
[CONFIG] SCVS_DIR : E:\TESIS

```

MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\data_super_compañias
[CONFIG] CLEAN_OUT: E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\outputs

```
[2]: # Funciones de normalización (self-contained)

def _strip_accents(s: str) -> str:
    return "".join(ch for ch in unicodedata.normalize("NFKD", s) if not
↳ unicodedata.combining(ch))

def normalize_company_name(value) -> str:
    if value is None: return ""
    if isinstance(value, float) and np.isnan(value): return ""
    s = str(value).strip()
    if not s: return ""
    s = _strip_accents(s); s = s.upper()
    s = re.sub(r"^[A-Z0-9 ]+", " ", s)
    s = re.sub(r"\b(SA|S\s*A|S\A\.|S\A\S\.|SAS|LTDA|CIA|C\.?IA\.?
↳ |COMPANIA|COMPAÑIA|CORP|INC|LLC|C\L\.|C\.?LTDA\.|&|Y)\b", " ", s)
    s = re.sub(r"\b(DE|DEL|LA|EL|LOS|LAS)\b", " ", s)
    return re.sub(r"s+", " ", s).strip()

def _normalize_ruc(value) -> str:
    if value is None: return ""
    if isinstance(value, float) and np.isnan(value): return ""
    return re.sub(r"D+", "", str(value).strip().replace(".0", ""))

def _norm_col_key(col: str) -> str:
    s = str(col or "").strip().upper()
    s = "".join(c for c in unicodedata.normalize("NFKD", s) if not unicodedata.
↳ combining(c))
    return re.sub(r"^[A-Z0-9 ]+", "", s)

# 1) Catálogo SCVS

def _pick_col(cols, patterns):
    cols_u = [str(c).upper() for c in cols]
    for pat in patterns:
        for i, cu in enumerate(cols_u):
            if pat in cu: return cols[i]
    return None

def _detect_excel_header_row(fp: Path, max_rows: int = 80) -> int:
    try: raw = pd.read_excel(fp, header=None, nrows=max_rows)
    except Exception: return 0
    best_i, best_score = 0, -1
    for i in range(len(raw)):
```

```

        vals = [v for v in row.iloc[i].tolist() if str(v).lower() != "nan"]
        if not vals: continue
        rt = " ".join(map(str, vals)).upper()
        sc = sum(tok in rt for tok in ["RUC", "IDENTIFIC"])*5 + sum(tok in rt
↪for tok in ["NOMBRE", "RAZON", "RAZÓN"])*3
        if sc > best_score: best_score = sc; best_i = i
    return int(best_i)

def _load_super_companies(folder: Path):
    if not folder.exists(): raise FileNotFoundError(f"No existe la carpeta SCVS:
↪ {folder}")
    files = sorted([p for p in folder.iterdir() if p.suffix.lower() in {".
↪xlsx", ".xls", ".csv"}])
    if not files: raise FileNotFoundError(f"Sin archivos en {folder}")
    frames = []
    for fp in files:
        try:
            df = pd.read_csv(fp) if fp.suffix.lower()==" .csv" else pd.
↪read_excel(fp, header=_detect_excel_header_row(fp))
            except Exception as e: print(f"[SCVS] {fp.name}: {e}"); continue
            if df is None or df.empty: continue
            df.columns = [str(c).strip() for c in df.columns]
            name_col = _pick_col(df.columns,
↪["RAZON", "RAZÓN", "NOMBRE", "DENOM", "COMPAÑ", "EMPRESA"])
            ruc_col = _pick_col(df.columns, ["RUC", "IDENTIFIC", "CEDULA"])
            if name_col is None:
                obj = [c for c in df.columns if str(df[c].dtype) in
↪("object", "string")]
                name_col = obj[0] if obj else None
            if ruc_col is None:
                best, bsc = None, -1.0
                for c in df.columns:
                    s = df[c].map(_normalize_ruc); sc = float((s!="").
↪mean()+3*float((s.str.len()==13).mean()))
                    if sc>bsc: bsc=sc; best=c
                ruc_col = best
            if name_col is None or ruc_col is None: continue
            tmp = pd.DataFrame({"super_name_raw": df[name_col].astype("string").
↪fillna("").map(str.strip), "super_ruc": df[ruc_col].map(_normalize_ruc)})
            tmp = tmp[tmp["super_ruc"].str.len()==13].copy()
            tmp["super_name_norm"] = tmp["super_name_raw"].
↪map(normalize_company_name)
            tmp = tmp[tmp["super_name_norm"].str.contains(r"[A-Z]", regex=True,
↪na=False) & (tmp["super_name_norm"].str.len()>=3)]
            if tmp.empty: continue
            frames.append(tmp[["super_name_raw", "super_name_norm", "super_ruc"]])

```

```

    if not frames: raise ValueError("No se pudo construir catálogo SCVS.")
    super_all = pd.concat(frames, ignore_index=True)
    super_best = (super_all.groupby(["super_name_norm", "super_ruc"],
    ↪as_index=False).size()
                  .sort_values(["super_name_norm", "size"],
    ↪ascending=[True, False])
                  .drop_duplicates("super_name_norm",
    ↪keep="first")["super_name_norm", "super_ruc"])
    return super_all, super_best

super_all, super_best = _load_super_companies(SCVS_DIR)
super_choices = super_best["super_name_norm"].dropna().tolist()
super_ruc_by_name = dict(zip(super_best["super_name_norm"],
    ↪super_best["super_ruc"]))
print(f"[SCVS] distinct nombres: {len(super_choices)}")

# Guardar como pickle (sin dependencia de pyarrow)
import pickle
ensure_dir(CLEAN_OUT)
with open(CLEAN_OUT / "super_best.pkl", "wb") as f: pickle.dump(super_best, f)
print(f"[OK] super_best.pkl guardado")

# 2) Catálogos SRI
SRI_CHUNK_ROWS = int(os.getenv("SRI_CHUNK_ROWS", "200000"))

def _sniff_sep(fp: Path) -> str:
    try: head = fp.open("rb").read(4096)
    except Exception: return "",
    counts = {"|": head.count(b"|"), ";": head.count(b";"), ",": head.
    ↪count(b","), "\t": head.count(b"\t")}
    sep = max(counts, key=counts.get)
    return sep if counts.get(sep, 0) > 0 else "",

def _pick_sri_columns(fp: Path):
    sep = _sniff_sep(fp)
    for enc in ["utf-8-sig", "utf-8", "cp1252", "latin1"]:
        try:
            hdr = pd.read_csv(fp, sep=sep, encoding=enc, dtype=str, nrows=0,
    ↪on_bad_lines="skip")
            norm_to_orig = {_norm_col_key(c): c for c in hdr.columns}
            ruc_col = next((norm_to_orig[a] for a in
    ↪["NUMERORUC", "NUMERODERUC", "RUC"] if a in norm_to_orig), None)
            razon_col = next((norm_to_orig[a] for a in
    ↪["RAZONSOCIAL", "RAZONSOC"] if a in norm_to_orig), None)

```

```

        fan_col = next((norm_to_orig[a] for a in
↪["NOMBREFANTASIACOMERCIAL", "NOMBREFANTASIA", "NOMBRECOMERCIAL"] if a in
↪norm_to_orig), None)
        if ruc_col and razon_col:
            return sep, enc, ruc_col, razon_col, fan_col
        except Exception: continue
        raise RuntimeError(f"No detecté columnas SRI en {fp.name}")

def _load_sri_catalogs(sri_dir: Path):
    files = sorted([p for p in sri_dir.iterdir() if p.suffix.lower()==".csv"])
    if not files: raise FileNotFoundError(f"Sin CSV en {sri_dir}")
    count_razon, count_fan = {}, {}
    n_rows = 0
    for i, fp in enumerate(files, 1):
        print(f"[SRI] ({i}/{len(files)}) {fp.name}")
        try: sep, enc, col_ruc, col_razon, col_fan = _pick_sri_columns(fp)
        except Exception as e: print(f" Saltado: {e}"); continue
        usecols = [c for c in [col_ruc, col_razon, col_fan] if c]
        try:
            for chunk in pd.read_csv(fp, sep=sep, encoding=enc, dtype=str,
↪usecols=usecols,
                                chunksize=SRI_CHUNK_ROWS, low_memory=True,
↪on_bad_lines="skip"):
                if chunk is None or chunk.empty: continue
                n_rows += len(chunk)
                rename = {col_ruc:"NUMERO_RUC", col_razon:"RAZON_SOCIAL"}
                if col_fan: rename[col_fan] = "NOMBRE_FANTASIA_COMERCIAL"
                chunk = chunk.rename(columns=rename)
                if "NOMBRE_FANTASIA_COMERCIAL" not in chunk.columns:
                    chunk["NOMBRE_FANTASIA_COMERCIAL"] = pd.NA
                chunk["NUMERO_RUC"] = chunk["NUMERO_RUC"].map(_normalize_ruc)
                # razón
                tmp = chunk[["NUMERO_RUC", "RAZON_SOCIAL"]].copy()
                tmp = tmp[tmp["RAZON_SOCIAL"].astype("string").notna()]
                tmp = tmp[tmp["RAZON_SOCIAL"].astype("string").str.strip() !=
↪""]

                if not tmp.empty:
                    tmp["name_norm"] = tmp["RAZON_SOCIAL"].
↪map(normalize_company_name)
                    tmp = tmp[(tmp["NUMERO_RUC"].str.len()==13) &
↪(tmp["name_norm"].str.len()>=3)]
                    for (nn, ruc), cnt in tmp.
↪groupby(["name_norm", "NUMERO_RUC"]).size().items():
                        key = (str(nn), str(ruc))
                        count_razon[key] = count_razon.get(key, 0) + int(cnt)
                    # fantasía

```

```

        tmpf = chunk[["NUMERO_RUC", "NOMBRE_FANTASIA_COMERCIAL"]].copy()
        tmpf = tmpf[tmpf["NOMBRE_FANTASIA_COMERCIAL"].astype("string").
↳notna()]

        tmpf = tmpf[tmpf["NOMBRE_FANTASIA_COMERCIAL"].astype("string").
↳str.strip() != ""]

        if not tmpf.empty:
            tmpf["fan_norm"] = tmpf["NOMBRE_FANTASIA_COMERCIAL"].
↳map(normalize_company_name)
            tmpf = tmpf[(tmpf["NUMERO_RUC"].str.len()==13) &
↳(tmpf["fan_norm"].str.len()>=3)]
            for (fn, ruc), cnt in tmpf.
↳groupby(["fan_norm", "NUMERO_RUC"]).size().items():
                key = (str(fn), str(ruc))
                count_fan[key] = count_fan.get(key,0) + int(cnt)
            except Exception as e:
                print(f" ERROR leyendo chunks: {e}")
            if not count_razon: raise ValueError("Catálogo SRI Razón vacío.")
            sri_razon_best = (
                pd.DataFrame([{"sri_razon_norm": k[0], "RUC": k[1], "freq": v} for k,v
↳in count_razon.items()])
                .sort_values(["sri_razon_norm", "freq"], ascending=[True, False])
                .drop_duplicates("sri_razon_norm",
↳keep="first")["sri_razon_norm", "RUC"]
                .reset_index(drop=True)
            )
            if count_fan:
                sri_fantasia_best = (
                    pd.DataFrame([{"sri_fantasia_norm": k[0], "RUC": k[1], "freq": v}
↳for k,v in count_fan.items()])
                    .sort_values(["sri_fantasia_norm", "freq"], ascending=[True, False])
                    .drop_duplicates("sri_fantasia_norm",
↳keep="first")["sri_fantasia_norm", "RUC"]
                    .reset_index(drop=True)
                )
            else:
                sri_fantasia_best = pd.DataFrame(columns=["sri_fantasia_norm", "RUC"])
                print(f"\n[SRI] Filas leídas: {n_rows}")
                print(f"[SRI] Razón distinct: {len(sri_razon_best)} Fantasia distinct:
↳{len(sri_fantasia_best)}")
                return sri_razon_best, sri_fantasia_best

sri_razon_best, sri_fantasia_best = _load_sri_catalogs(SRI_DIR)
sri_razon_choices = sri_razon_best["sri_razon_norm"].dropna().tolist()
sri_razon_ruc_by_name = dict(zip(sri_razon_best["sri_razon_norm"],
↳sri_razon_best["RUC"]))
sri_fan_choices = sri_fantasia_best["sri_fantasia_norm"].dropna().tolist()

```

```
sri_fan_ruc_by_name = dict(zip(sri_fantasia_best["sri_fantasia_norm"],
↪sri_fantasia_best["RUC"]))

with open(CLEAN_OUT / "sri_razon_best.pkl", "wb") as f: pickle.
↪dump(sri_razon_best, f)
with open(CLEAN_OUT / "sri_fantasia_best.pkl", "wb") as f: pickle.
↪dump(sri_fantasia_best, f)
print(f"[OK] sri_razon_best.pkl y sri_fantasia_best.pkl guardados")
```

```
[SCVS] distinct nombres: 214446
[OK] super_best.pkl guardado
[SRI] (1/26) SRI_Catastro_Empresas_Fantasmas.csv
[SRI] (2/26) SRI_MERCADESENLINEA.csv
[SRI] (3/26) SRI_RUC_Azuay.csv
[SRI] (4/26) SRI_RUC_Bolivar.csv
[SRI] (5/26) SRI_RUC_Carchi.csv
[SRI] (6/26) SRI_RUC_Cañar.csv
[SRI] (7/26) SRI_RUC_Chimborazo.csv
[SRI] (8/26) SRI_RUC_Cotopaxi.csv
[SRI] (9/26) SRI_RUC_El_Oro.csv
[SRI] (10/26) SRI_RUC_Esmeraldas.csv
[SRI] (11/26) SRI_RUC_Galapagos.csv
[SRI] (12/26) SRI_RUC_Guayas.csv
[SRI] (13/26) SRI_RUC_Imbabura.csv
[SRI] (14/26) SRI_RUC_Loja.csv
[SRI] (15/26) SRI_RUC_Los_Rios.csv
[SRI] (16/26) SRI_RUC_Manabi.csv
[SRI] (17/26) SRI_RUC_Morona_Santiago.csv
[SRI] (18/26) SRI_RUC_Napo.csv
[SRI] (19/26) SRI_RUC_Orellana.csv
[SRI] (20/26) SRI_RUC_Pastaza.csv
[SRI] (21/26) SRI_RUC_Pichincha.csv
[SRI] (22/26) SRI_RUC_Santa_Elena.csv
[SRI] (23/26) SRI_RUC_Santo_Domingo.csv
[SRI] (24/26) SRI_RUC_Sucumbios.csv
[SRI] (25/26) SRI_RUC_Tungurahua.csv
[SRI] (26/26) SRI_RUC_Zamora_Chinchipe.csv

[SRI] Filas leídas: 8051635
[SRI] Razón distinct: 6653039  Fantasía distinct: 1469998
[OK] sri_razon_best.pkl y sri_fantasia_best.pkl guardados
```

1.4 Migrate Here From Source Notebook

- Centralized SRI and SCVS catalog loading.
- Leads fuzzy tournament.
- Horas fuzzy tournament.

1.4.1 Suggested Source Cells

- Code cells: 23, 24, 25

```
[3]: # Torneo Fuzzy (batch vectorizado)

leads_ruc_exact = read_csv_checked(CLEAN_OUT / "leads_ruc_exact.csv")
horas_ruc_exact = read_csv_checked(CLEAN_OUT / "horas_ruc_exact.csv")

UMBRAL = int(os.getenv("UMBRAL_SRI", "80"))
_PRIO = {"SCVS": 0, "SRI_RAZON": 1, "SRI_FANTASIA": 2}

try:
    from rapidfuzz import process as _rfp, fuzz as _rff
    _HAVE_RF = True
    print("[TORNEO] Motor: rapidfuzz (batch cdist, workers=-1)")
except ImportError:
    _HAVE_RF = False
    print("[TORNEO] Motor: difflib (fallback secuencial - instala rapidfuzz para mayor velocidad)")

def _run_tournament(unmatched: pd.DataFrame, q_col: str, raw_col: str) -> pd.DataFrame:
    COLS = [raw_col, q_col, "source_winner", "winner_name_norm", "score", "RUC"]

    queries = unmatched[q_col].fillna("").astype(str).str.strip().tolist()
    raw_vals = unmatched[raw_col].tolist() if raw_col in unmatched.columns else [""] * len(queries)
    if not any(queries):
        return pd.DataFrame(columns=COLS)

    catalogs = [
        (super_choices, super_ruc_by_name, "SCVS"),
        (sri_razon_choices, sri_razon_ruc_by_name, "SRI_RAZON"),
        (sri_fan_choices, sri_fan_ruc_by_name, "SRI_FANTASIA"),
    ]

    n = len(queries)
    best_scores = np.full(n, -1.0, dtype=np.float32)
    best_idx = np.full(n, -1, dtype=np.int32)
    best_srcs = [""] * n

    if _HAVE_RF:
        # Modo rápido: cdist calcula toda la matriz en C con paralelismo real
        CHUNK = 500_000 # bloques para no saturar RAM con catálogos muy grandes
        for choices, _, src in catalogs:
            if not choices: continue
            for start in range(0, len(choices), CHUNK):
```

```

sub = choices[start:start + CHUNK]
mat = _rfp.cdist(queries, sub,
                 scorer=_rff.token_set_ratio,
                 score_cutoff=UMBRAL - 1,
                 workers=-1,
                 dtype=np.float32)
sub_idx = mat.argmax(axis=1)
sub_sc = mat[np.arange(n), sub_idx]
sub_glob = np.array(sub_idx + start, dtype=np.int32)

improve = (sub_sc >= UMBRAL) & (
    (sub_sc > best_scores) |
    ((sub_sc == best_scores) &
     np.array([_PRIO[src] < _PRIO.get(best_srcs[i], 99) for i in
↪in range(n)]))
)
best_scores = np.where(improve, sub_sc, best_scores)
best_idxes = np.where(improve, sub_glob, best_idxes)
best_srcs = [src if improve[i] else best_srcs[i] for i in
↪range(n)]
else:
    # Fallback difflib
    from difflib import SequenceMatcher
    for choices, _, src in catalogs:
        if not choices: continue
        for i, q in enumerate(queries):
            if not q: continue
            best_sc, best_j = -1.0, -1
            for j, c in enumerate(choices):
                sc = SequenceMatcher(None, q, c).ratio() * 100
                if sc > best_sc: best_sc, best_j = sc, j
            if best_sc >= UMBRAL and (best_sc > best_scores[i] or
↪(best_sc == best_scores[i] and _PRIO[src] < _PRIO.
↪get(best_srcs[i], 99))):
                best_scores[i] = best_sc
                best_idxes[i] = best_j
                best_srcs[i] = src

ruc_maps = {"SCVS": super_ruc_by_name, "SRI_RAZON": sri_razon_ruc_by_name,
↪ "SRI_FANTASIA": sri_fan_ruc_by_name}
cat_lists = {"SCVS": super_choices, "SRI_RAZON": sri_razon_choices,
↪ "SRI_FANTASIA": sri_fan_choices}

rows = []
for i in range(n):
    q = queries[i]
    if not q or best_scores[i] < UMBRAL: continue

```

```

src_win    = best_srcs[i]
best_name  = cat_lists[src_win][best_idx[i]]
ruc        = ruc_maps[src_win].get(best_name, "")
rows.append({raw_col: raw_vals[i], q_col: q, "source_winner": src_win,
             "winner_name_norm": best_name, "score":
↪float(best_scores[i]),
             "RUC": str(ruc) if ruc else ""})

if not rows:
    return pd.DataFrame(columns=COLS)
return pd.DataFrame(rows).sort_values("score", ascending=False).
↪reset_index(drop=True)

unmatched_leads = leads_ruc_exact[leads_ruc_exact["RUC"].isna()].copy()
unmatched_horas = horas_ruc_exact[horas_ruc_exact["RUC"].isna()].copy()
print(f"[TORNEO] Leads huérfanas: {len(unmatched_leads)} Horas huérfanas:
↪{len(unmatched_horas)}")

leads_torneo_ganadores = _run_tournament(unmatched_leads, "Company_norm",
↪"Company_raw")
horas_torneo_ganadores = _run_tournament(unmatched_horas, "EMPRESA_norm",
↪"EMPRESA_raw")

print(f"\n[LEADS/TORNEO] Ganadores: {len(leads_torneo_ganadores)}")
if not leads_torneo_ganadores.empty:
    print(leads_torneo_ganadores["source_winner"].value_counts().to_string())
print(f"\n[HORAS/TORNEO] Ganadores: {len(horas_torneo_ganadores)}")
if not horas_torneo_ganadores.empty:
    print(horas_torneo_ganadores["source_winner"].value_counts().to_string())

_ = save_df_csv(leads_torneo_ganadores, CLEAN_OUT / "leads_torneo_ganadores.
↪csv")
_ = save_df_csv(horas_torneo_ganadores, CLEAN_OUT / "horas_torneo_ganadores.
↪csv")

display(leads_torneo_ganadores.head(10))

```

[TORNEO] Motor: rapidfuzz (batch cdist, workers=-1)

[TORNEO] Leads huérfanas: 320 Horas huérfanas: 101

[LEADS/TORNEO] Ganadores: 305

source_winner

SRI_FANTASIA 154

SCVS 78

SRI_RAZON 73

[HORAS/TORNEO] Ganadores: 99

source_winner

SCVS 39

SRI_FANTASIA 31

SRI_RAZON 29

[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning
\outputs\leads_torneo_ganadores.csv shape: (305, 6)

[OK] Guardado: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning
\outputs\horas_torneo_ganadores.csv shape: (99, 6)

	Company_raw	Company_norm	source_winner \
0	WWE	WWE	SCVS
1	THE HOME DEPOT MÉXICO	THE HOME DEPOT MEXICO	SCVS
2	Tesalia	TESALIA	SCVS
3	Telefonica	TELEFONICA	SCVS
4	TECNOLOGICO DE MONTERREY	TECNOLOGICO MONTERREY	SRI_RAZON
5	TALMEX PHARMA	TALMEX PHARMA	SRI_FANTASIA
6	Talma	TALMA	SCVS
7	SYM	SYM	SRI_RAZON
8	SUMA GLOBAL PACK	SUMA GLOBAL PACK	SRI_RAZON
9	STEVE MADDEN	STEVE MADDEN	SCVS

	winner_name_norm	score	RUC
0	WORLD WIDE ELECTRIC SUPPLY WWE ECUADOR S EN LI...	100.0	1793160182001
1	DEPOT	100.0	0991307281001
2	THE TESALIA SPRINGS COMPANY	100.0	1790005739001
3	ASISTENCIA TECNICA TELEFONICA ASSISTPHONE	100.0	1792712750001
4	ASOCIACION CIVIL INSTITUTO TECNOLOGICO ESTUDIO...	100.0	0991463712001
5	PHARMA	100.0	0910479948001
6	TALMA ECUADOR SERVICIOS AEROPORTUARIOS	100.0	1791807162001
7	CONSORCIO SYM CONTABLES TRIBUTARIOS	100.0	1792585376001
8	GLOBAL	100.0	1792923611001
9	STEVE MADDEN DISTRIBUTION ECUADOR S	100.0	1793216824001

[4]: # Verificación final

```
import pickle
```

```
required = [
```

```
    CLEAN_OUT / "super_best.pkl",
```

```
    CLEAN_OUT / "sri_razon_best.pkl",
```

```
    CLEAN_OUT / "sri_fantasia_best.pkl",
```

```
    CLEAN_OUT / "leads_torneo_ganadores.csv",
```

```
    CLEAN_OUT / "horas_torneo_ganadores.csv",
```

```
]
```

```
for p in required:
```

```
    if not p.exists(): raise FileNotFoundError(f"Output faltante: {p}")
```

```
    if p.suffix == ".pkl":
```

```
        with open(p, "rb") as f: df_chk = pickle.load(f)
```

```
else:
    df_chk = pd.read_csv(p)
    print(f"[OK] {p.name:45s} shape={df_chk.shape}")
print("\n Notebook 03_catalogos_y_torneo_fuzzy completado correctamente.")
```

```
[OK] super_best.pkl shape=(214446, 2)
[OK] sri_razon_best.pkl shape=(6653039, 2)
[OK] sri_fantasia_best.pkl shape=(1469998, 2)
[OK] leads_torneo_ganadores.csv shape=(305, 6)
[OK] horas_torneo_ganadores.csv shape=(99, 6)
```

Notebook 03_catalogos_y_torneo_fuzzy completado correctamente.